

Clawbada Character Animation System — Designer Handoff

What This Document Covers

This spec explains how Clawbada’s lobster characters will be animated for the live game using Unity’s 2D rigging tool chain. It’s a handoff ready for implementation after the battle board design work wraps up.

The core problem it solves: **breeding produces thousands of unique body-part combinations, and we can’t hand-draw animation frames for every possible lobster.** This system lets one animation per class work for every variant of that class, without multiplying the art budget.

What Changed and Why

We previously explored building frame-by-frame sprite sheets per lobster combination. That approach scales badly: 10 classes x many body-part variants x multiple animations each = tens of thousands of frames, most of which players will never notice at battle viewing distance.

The shift: we’re moving to Unity’s 2D skeletal animation tool chain. Unity rigs bones to sprite parts and animates the bones — meaning one animation clip plays correctly across every body-part combination without redrawing anything.

What stays hand-drawn: class rigs (one per class), individual body-part variant sprites (single PNGs, not sheets), and tier treatment overlays for Elite and Apex.

What Unity handles: frame interpolation, movement, attack timing, rigged skinning, runtime sprite swapping based on the lobster’s DNA.

This approach preserves the carefully designed class palettes, respects the “class identity first” principle, and keeps the art budget bounded even as the breeding population explodes.

The Core Principle: Rig Owns Motion, Sprite Owns Appearance

Think of each class as a **flipbook with swappable stickers**:

- **The flipbook** is the class animation — the timing, pose sequence, and feel. Leviathan’s flipbook flips slow and heavy. Mantis’s flips fast and snappy. That motion signature is baked into how the designer keyframes the rig, and it’s what makes each class feel distinct on screen.

- **The stickers** are the body-part variants — curved claws, scythe claws, spindly legs, thick legs. Each variant is a single sprite that rides on top of the rig, occupying a specific slot. Breeding swaps which sticker goes into each slot.

Because the flipbook animates the **bones** (not the pixels), every sticker combo plays the animation correctly. A Mantis with scythe claws and a Mantis with pincer claws use the identical attack animation — the bones move the same way, but the visual result is different because the sprite swapped.

Design implication: you animate each class once. The visual variation from breeding comes from drawing multiple variant sprites, not from animating each combination.

Unity Tool Chain

Unity's 2D Animation package handles this workflow end-to-end. The pieces you'll use:

Tool	What It Does
2D Animation package	Install via Package Manager. Contains all the rigging tools.
PSD Importer	Imports layered PSD files — each layer becomes a sprite, auto-ready for rigging.
Sprite Skin component	Binds sprite parts to bones so the sprite deforms with bone motion.
Sprite Library Asset	Registers sprite variants under category/label names (e.g. claws/scythe).
Sprite Resolver component	Runtime swap — picks a sprite from the library based on DNA at spawn time.
Unity Animator	Authors animation clips and state machines (idle to walk to attack, etc.).
2D IK (optional)	Inverse kinematics for leg articulation during dashes or uneven terrain.

You already have body parts per class as separate files — those are the starting point. PSD Importer can pull them in directly if we combine them into a layered PSD per class.

Deliverable 1: Class Rigs (10 total)

One rig per class. Each rig contains:

- **Bones** positioned through the body — spine, claws (left and right), leg chain, tail segments, antenna, carapace, eyes
- **Skinned sprites** — each body part sprite is bound to its bone or bones
- **Pivot anchors** per body-part slot — the attachment points variants will swap into (see Pivot Convention below)

Classes (all 10 finalized):

Class	Motion feel	Move range
Bulwark	Slow, planted, heavy	1 hex
Leviathan	Slow, lumbering, massive	1 hex
Sentinel	Moderate, supportive, poised	2 hexes
Abyss	Moderate, predatory	2 hexes
Kraken	Moderate, controlled	2 hexes
Reaver	Moderate-fast, aggressive	2 hexes
Mantis	Fast, darting, ninja-like	3 hexes
Tempest	Fast, stormy, unpredictable	3 hexes
Specter	Fast, ghostly, floaty	3 hexes
Ember	Fast, scurrying, fragile	3 hexes

The motion feel for each class should be baked into the rig's keyframe timing. This is where the class identity lives — breeding can't override it.

Deliverable 2: Animation Clips (~8 per class, 80 total)

For each class rig, author these animation clips:

Clip	Purpose	Duration (approx)
Idle	Breathing/bob loop when not acting	2 sec loop
Walk	Movement between hexes	0.4 to 0.6 sec loop
Attack	Primary attack swing	0.4 to 0.7 sec
Defend	Defensive stance and brace	0.5 to 1.0 sec
Special	Class-specific special move	0.8 to 1.5 sec
Damage	Hit reaction	0.3 sec
Death	Defeat animation (plays once)	1.0 to 1.5 sec
Victory	Post-battle pose	1.5 sec loop

Clips keyframe **bones**. Every body-part variant uses these clips without modification.

Special move per class — the Special clip needs to express the class's signature ability:

Class	Special	Visual direction
Bulwark	Fortify	Team-wide defensive stance, shield glow
Mantis	Ambush	Flash-dash into target, ignore-armor strike
Leviathan	Crush	Massive two-claw slam, single target
Tempest	Maelstrom	AoE whirlwind from body, 3-hex radius
Specter	Haunt	Ranged spectral projection, debuff
Sentinel	Rally	Heal aura on nearby ally
Reaver	Rend	Close-in claw slash with bleed effect
Abyss	Devour	Melee lifesteal bite
Kraken	Bind	Mid-range tentacle grab, stun
Ember	Inferno	Max-range fire projectile, self-damage

Deliverable 3: Variant Sprites (~170 total, class-locked)

Each body part gets multiple variants per class — drawn in that class’s exact palette. **Do not share variant sprites across classes via color remapping.** The class palettes are identity design, and palette shifting at scale dilutes that identity.

Variant count per class (S1 ship target):

Part	Variants per class	x 10 classes	Role in animation
Claws	4	40	Animated — swings with attack and special
Legs	3	30	Animated — walk cycle and idle stance
Tail	2	20	Animated — dash and counterbalance
Carapace	3	30	Static overlay — shell/back visual
Eyes	3	30	Static overlay (free blink via rig bone)
Antennae	2	20	Static overlay (free twitch via rig bone)
Total	17	~170 sprites	

Per-class visual diversity: $4 \times 3 \times 2 \times 3 \times 3 \times 2 = 432$ **unique appearances per class.** Across the 10-class roster, that’s **4,320 total lobster appearances** before tier treatments are layered on top. Plenty for breeding to feel meaningful at launch.

Why claws get the most variants: claws are on screen during every attack and special move, and their silhouette (scythe vs pincer vs hammer vs muscle-bulge) is what players read most at battle viewing distance. Prioritize claw variety.

Format per variant:

- Single PNG file (no sprite sheet — animation is bone-driven)

- Transparent background
- Drawn in the class's exact palette
- Anchored to the rig's pivot (see Pivot Convention below)
- Same canvas size as the class template for that part

Pivot / Anchor Convention

Every variant of a given part must attach to the rig at the **same pivot point**. If scythe claws and pincer claws have different pivots, swapping them at runtime will visibly pop on screen.

To lock before drawing variants:

1. Per class, per part slot, define a pivot point in pixel coordinates (relative to the sprite canvas).
2. Draw all variants of that slot with the pivot at that exact location.
3. Document pivot coordinates in a reference file alongside the assets.

This is a one-time setup per class but it's critical. A pivot-check validation tool can flag variants with inconsistent anchors if that's useful.

Tier Treatments: Evolved / Elite / Apex

Battle-playable tiers are Evolved, Elite, and Apex only. Base tier lobsters cannot enter battle — they only appear in mining, marketplace, breeding UI, and team builder screens. Base tier needs simpler art (idle pose plus a simple walk cycle) and does not need the full combat animation set.

Tier visual escalation (applied on top of the class rig and variants):

Tier	Scale	Overlay	Palette shift	VFX
Evolved	100% (baseline)	none	class default	minimal
Elite	108%	armor plating sprite	richer metallics	attack parti- cles, body aura
Apex	115%	heavy plating and spikes	premium shift	full aura, trail, screen FX on specials

Tier treatments are per-class, not per-variant. Each class gets its own Elite and Apex overlay set (armor pieces, particle prefabs, palette modifier). Those overlays appear on top of whatever variants the lobster has, so a player's specific variant choices aren't lost — they just get “powered up” visually.

Tier budget (approximate):

- 10 classes x 2 tiers (Elite + Apex) x ~4-6 overlay pieces per tier = **80 to 120 overlay sprites**
- Plus Unity particle system prefabs per tier (glows, trails, auras)
- Optional: ~20 Apex-exclusive hero key frames (10 classes x 2 signature poses: new idle pose and new special hero pose) for trophy payoff

Base tier (non-battle) budget, separately:

- 10 classes x 1 idle pose + 1 simple walk cycle ~ 20 sprites total
- Used only in marketplace thumbnails, team builder, mining dashboard, breeding UI

Claws: Special Case for Pincer Motion

Claws likely need to **open and close** during attacks. Two approaches to consider:

Option A (simpler, recommended for S1):

- Each claw variant has two states: open and closed
- Sprite Resolver swaps between them at specific animation keyframes
- Register as `claws/scythe_open` and `claws/scythe_closed` in the Sprite Library

Option B (more flexible, higher effort):

- Each claw variant is a sub-rig with its own internal bones
- Sprite Skin deforms the claw open/closed via bone rotation
- One sprite per variant, smoother motion

Start with Option A. Ship the simpler system and upgrade to sub-rigs later if the motion feels too stiff in playtest.

The Readability Rule

Every variant must be **silhouette-distinct at battle viewing distance**. The hex grid is 6x5 on a 480x270-scale canvas, so lobsters end up roughly 64-96 pixels tall on screen.

What reads at that scale:

- Silhouette changes (claw shape, leg profile, tail fan)
- Color blocks (the class palette's three triplets)
- Motion signature (gait, swing arc)
- Large VFX (glows, particles, trails)

What doesn't read at that scale:

- Fine pixel detail inside a part
- Subtle palette variations
- Small idle twitches on small parts

Practical guidance: sketch variants along silhouette axes first (thick/thin, long/short, spiky/smooth), then add surface detail. If two variants only differ in fine detail, they'll look identical on the hex grid — that's wasted art budget.

Static Overlay Parts (Carapace, Eyes, Antennae)

These parts are drawn as single sprites and don't need per-variant animation frames. BUT the class rig can add **free micro-motion** for them via bone keyframes:

- **Eye blink** — rotate or scale the eye bone briefly in the idle clip. One keyframe per class, works for every eye variant.
- **Antenna twitch** — small bone rotation on the antenna bones during idle.
- **Carapace bob** — the spine bone bobs gently for breathing, naturally lifting the carapace.

None of these require per-variant work. They're class-rig keyframes that stamp the static sprite into a slightly different position per frame.

Expansion Runway

The on-chain DNA schema allows **up to 16 variants per body part** (4-bit variant field). S1 ships with 4 max variants per part per class, but we can grow that over time without any contract or data schema changes. New variants just drop into the Unity Sprite Library as additional entries.

Recommended expansion cadence:

- **S1 launch** — 4 claws, 3 legs, 2 tail, 3 carapace, 3 eyes, 2 antennae per class (17 total per class)
- **S2 expansion** — add 2-4 variants to the parts players care about most (probably claws first)
- **S3 and beyond** — expand toward the full 16 per part if engagement supports it

Prerequisites Before Starting

1. **Battle board designs complete** — current work, blocking this system.
 2. **Pivot/anchor convention locked** per class, per part slot, with pivot coordinates documented.
 3. **Unity project setup** — install 2D Animation package plus PSD Importer.
 4. **Class rig template** — the first class rig serves as the reference; subsequent classes follow its bone layout and naming.
-

Delivery Format

- **Rigs** — Unity .prefab files, one per class, in `Assets/Characters/Rigs/`
 - **Animation clips** — .anim files per class, organized in `Assets/Characters/Animations/<ClassName>/`
 - **Variant sprites** — PNG files organized by class and part: `Assets/Characters/Variants/<ClassName>/`
 - **Sprite Libraries** — one .spriteLib per class, registering all variants under category/label keys
 - **Tier overlays** — per-class overlay sprites in `Assets/Characters/TierOverlays/<ClassName>/<Tier>/`
 - **Particle prefabs** — .prefab files in `Assets/Characters/VFX/Tier/`
-

Open Questions for Discussion

1. **Should Apex get hand-drawn signature key frames** (unique idle pose plus unique special pose) beyond overlay treatments, given it's the trophy tier players spend the most to reach?
2. **Claws Option A vs B** — which feels better in a first prototype pass?
3. **Pivot validation tool** — worth building before drawing many variants, or check manually for S1?
4. **Animation pacing** — should fast classes (3-hex move range) have noticeably shorter clip durations than slow classes (1-hex), or does movement speed on the hex grid communicate that enough?

These are design decisions to make during the first class rig pass, not blockers for starting work.

What This Supersedes

This system replaces the earlier hand-drawn sprite sheet approach discussed in initial animation conversations. It also supersedes the use of

`packages/asset-gen/tools/animation-rig-v2.html` for character animation in the live game — that tool remains useful for battle simulation prototyping and VFX authoring, but character animation for production moves to Unity's 2D rigging pipeline.

End of spec. Questions welcome — this is a starting framework, not a locked contract.